Project concepts

The project explained from zero: ideas, data, results, limits

CNN-LSTM Based Spatiotemporal Flood Evolution Forecasting · Anhad Mahajan and Aman Sumeet Saxena · Deep Learning CA3

1 The whole project in five minutes

When a river overflows or heavy rain falls, people need to know where the water will be in the next few hours. A **physics simulator** can work this out but is slow (the dataset makers report days for one large flood). We teach a **neural network** to do the same job quickly. We show it the last few depth maps (each a map of the water depth in every little square of land), plus the rain and the ground height, and ask it to draw the *next* map.

We used the public dataset **FloodCastBench**: four floods re-created in a simulator from real ground heights, land maps and satellite rain, in Australia, the UK, Pakistan and Mozambique. The network learns from Australia and Pakistan. The UK and Mozambique stay hidden until the final test, to see whether it works on a place it has never met.

The key question. In 30 minutes the water in most places hardly changes, so even "copy the last map" is a very good forecast. A model is only worth using if it beats simple guesses. The score is called **skill**: 0 means no better than copying and 1 means perfect.

1. **Model A** (CNN-LSTM, Colab) beat copying in 3 of 4 tests, and at first we were happy.
2. The **trend forecast** (take the last map and keep most of the last 30-minute change going, one fitted number, no network) scored 0.91 to 0.99, while Model A scored only 0.15 to 0.35 where it beat copying. In late Australia Model A was *worse* than copying (skill -1.449).
3. **Model B** starts from the trend and learns a small correction. It matched the trend but did not beat it, because only 1% to 9% of the error of copying is left after the trend.
4. A laptop study (about 310 training runs, up to 2.5 hours ahead) found a linear formula scoring 0.79 to 0.82 (after a bug fix, Chapter 14) against 0.81 to 0.87 for the best networks. The LSTM memory did not help at equal training.
5. On top we built a **capsule network** that rates the flooding, **written rules** that check it (*NeuroSymbolic AI*) and **Grad-CAM** heat maps (*explainable AI*).

The lesson: **always compare a model with simple forecasts before you trust it.**

Table 1. The two parts of the project. Their numbers must never be compared with each other.

	Part 1: Colab notebook	**Part 2: laptop study**
Where it ran	Google Colab, GPU	One laptop, CPU
How far ahead	One map, 30 minutes	Five maps in a row, up to 2.5 hours
Input size	Whole maps (learns on 64 × 64 tiles)	Patches of 128 × 128 squares
Models and runs	Model A, Model B, trend, capsule, rules, Grad-CAM. One run each	CNN-LSTM, local CNN, ConvLSTMs, simple baselines. About 310 runs

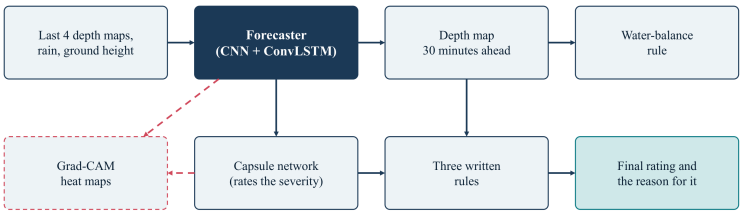


Figure 1. Everything we built. The forecaster is Chapters 4 to 6 and 10, the capsule network and rules Chapter 15, Grad-CAM (dashed arrows) Chapter 16.

2 Floods, forecasts and the long title

A **forecast** is a careful guess about the future. The simulator is detailed but slow, a trained network answers in seconds. The title decodes as: **CNN** (convolutional neural network, looks at pictures by sliding a small window, Chapter 5), **LSTM** (long short-term memory, reads things in order and remembers, Chapter 6), **spatiotemporal** (space and time) and **flood evolution** (how a flood grows and changes). So: *use a network that sees pictures and remembers the past to forecast how flood water spreads in space as time passes.* It is a careful test of how much a CNN-LSTM adds over simple guesses, not a ready warning system, because the floods are simulated and the models learned to imitate the simulator.

3 A picture is just a grid of numbers

A flood map is a **grid** of small squares, each holding one number: the water depth in metres (0.0 is dry). In Australia and the UK a square is **30 m** wide, in Pakistan and Mozambique **480 m** (16 times wider). The Australia map has 1073 × 1073 squares. We **never mix the two sizes**: one model for each.

For every square we also know the ground height, the rain and the land cover, like clear sheets stacked on one another. Each sheet is a **channel**. The model uses three: water depth, rain and a version of the ground height. Depth is stored every 5 minutes but rain only every 30, so we keep **one map every 30 minutes**, like a flipbook (the 10-day Australia flood becomes 481 maps). Depth is in metres, rain in mm per hour, and errors in centimetres.

4 What is a neural network, and how does it learn?

A programmer normally writes rules, which is nearly impossible for water. A **neural network** is given many *examples* and finds the rules itself: it **guesses, sees how wrong it was, and adjusts**. It is built from **neurons**. A neuron multiplies each input by its own **weight**, adds the results and applies a simple rule, **ReLU**: if the number is negative make it zero, otherwise leave it. ReLU adds the bend, without which many layers would be

no better than one weighted sum. Many neurons side by side form a **layer**. The weights plus one bias per neuron are the **parameters**. Our main model has **62,833**.

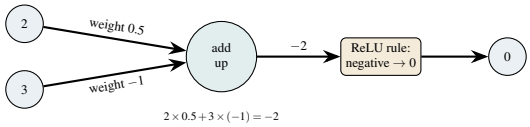


Figure 2. One neuron with made-up numbers. The sum -2 is negative, so ReLU turns it into 0.

The learning loop. Weights start random. Then the network repeats: **guess**, **measure** how wrong (the loss), **nudge** every weight a little. The **loss** is the **mean squared error**: square each mistake and average (squaring stops -0.1 and $+0.1$ cancelling and punishes big mistakes more). To know which way to nudge, the network uses the **gradient**, the slope of the mistake: like standing on a foggy hill, feeling which way the ground slopes and stepping downhill. The step size is the **learning rate**, the method that decides each step is the **optimiser**. **Adam** is a popular optimiser that adapts its step size, and we used it throughout with learning rate 0.001. The network nudges after a small group of examples, a **batch** (16 in Colab, 1 on the laptop). One trip through all training examples is an **epoch** (we call it a pass).

Overfitting and early stopping. A network can memorise its practice examples and fail on anything new. This is **overfitting**. We split each flood in time order: training (first 70%, learned from), validation (15%, checked during training) and test (last 15%, used only at the end). A never-seen flood is an even harder exam. The sign of overfitting is practice error falling while validation error rises, and **early stopping** stops when validation stops improving and keeps the best version (used in Part 1). We meet a real case in Chapter 11.

Models are written in **Python** with **TensorFlow** and **Keras**. A **GPU** does the many tiny calculations much faster than a **CPU**. Part 1 ran on a Colab GPU and Part 2 on one laptop CPU.

5 Teaching a network to look at maps: the CNN

A **convolutional neural network** (**CNN**) uses a small window called a **filter** (or **kernel**), commonly 3×3. We lay it over one spot, multiply the 9 numbers under it by 9 weights and add them: one number saying "how much does this spot look like my pattern?" Then we slide it one step (the **stride** is 1) until the map is covered, which gives a **feature map**. Adding a border of zeros (**padding**) keeps the map the same size. The **same filter is used everywhere**, so the parameter count stays small.

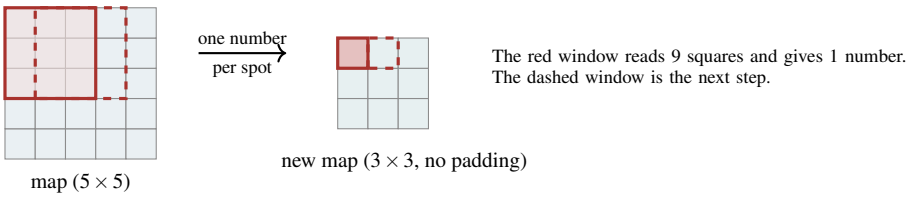


Figure 3. A filter slides over the map. Every spot gets one number.

One filter finds one pattern, so a layer uses many (for example 16), giving 16 feature maps that are the 16 *channels* the next layer reads. A second convolution layer combines the simple patterns of the first (edges, slopes) into bigger ones such as a river bend, and each square sees a larger area. After each filter we apply ReLU. (Pooling is not used.) **Real count:** the first layer has 16 filters of 3×3 reading 3 channels, so $3 \times 3 \times 3 = 27$ weights plus 1 bias is 28 per filter and $16 \times 28 = \mathbf{448}$ parameters. The second reads 16 channels: $3 \times 3 \times 16 + 1 = 145$ per filter and $16 \times 145 = \mathbf{2{,}320}$.

6 Teaching a network to remember: LSTM and ConvLSTM

One flood map cannot tell you whether the water is rising, you need the map before it. An **LSTM** (*long short-term memory*) reads one step at a time and keeps a memory, like a student with a notebook who at each step does three things: **forget** (erase what no longer matters), **write** (add what is useful) and **read** (look in the notebook to answer). Small dials called **gates** (forget, input, output) do these jobs, and the network learns how far to turn each. A normal LSTM keeps its memory as a short list of numbers. Ours must say something about *each place*, so we use a **ConvLSTM**: its memory is a map and its gates use sliding filters like the CNN, so each square is updated using its neighbours too.

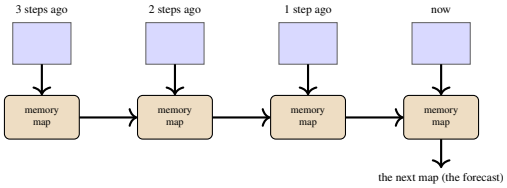


Figure 4. A ConvLSTM reads the maps in order and updates a memory that is itself a map.

Our ConvLSTM has 32 filters, and 55,424 of the 62,833 parameters live here. The big question of the project is: **does the memory actually help, or is a simpler method just as good?**

7 The data: four floods

FloodCastBench (Xu and co-authors, *Scientific Data*, 2025) re-created four real floods in a physics simulator using real **ground height** (FABDEM), **land cover** and satellite **rain** (GPM-IMERG). Depth is given every 5 minutes, rain every 30. The maps are **simulated**, so our network learns to imitate the simulator, not real flooding.

Australia and the UK also come at 60 m (used by the laptop study), made by resizing the 30 m results. Two pairs: learn from **Australia**, test on the **UK**, and learn from **Pakistan**, test on **Mozambique**.

Table 2. The four floods. "Maps" is the number of depth maps we keep, one every 30 minutes.

Flood	Dates	Square	Map size	Length	Maps	Our role
Australia	20 Feb to 2 Mar 2022	30 m	1073 × 1073	10 days	481	Training
UK	4 to 7 Dec 2015	30 m	170 × 275	3 days	145	Never-seen test
Pakistan	18 to 31 Aug 2022	480 m	810 × 441	14 days	673	Training
Mozambique	14 to 20 Mar 2019	480 m	151 × 138	6 days	289	Never-seen test

Getting the data ready. (1) One map every 30 minutes. (2) One common grid: two independent checks (rivers of the land cover map and valleys of the ground height map against the water) agreed within about one square, then small squares were averaged into big ones. (3) Rain timing: rain file j holds the rain in the 30 minutes *after* depth map j, and on rainy steps the measured rise was 0.494 to 0.502 times the file value in all four floods, as expected for half an hour. (4) Three rain settings: **causal** (only past rain), **hybrid** (the original input, with up to 30 minutes of rain not yet fallen) and **forced** (the next five rain maps also given). (5) Local ground height: the height of a square minus the average around it (15 × 15 in Colab, the 128 × 128 patch on the laptop), because water flows by height differences. (6) **Tiles** (patches) of 64 × 64 in Colab and 128 × 128 on the laptop, because a million squares is too much in one piece. A trained network can then run on a whole map.

8 The task and how we keep score

Part 1: four depth maps 30 minutes apart plus rain and local height in, **one** map 30 minutes later out. **Part 2:** five maps in, the next five out (up to 2.5 hours) in one go.

The simple forecasts we compare with. *Copy the last map* (persistence, skill 0 by definition), the *trend forecast*, a *linear model* of the depth history (fitted weighted sum), a *lookup table* (average answer for each pair of current depth and last change) and a *per-pixel MLP* (a tiny network looking at one square). The last three are Part 2 only. With d_t the depth now and d_{t-1} 30 minutes ago,

$$\text{trend forecast} = \max\big(0,\ d_t + \beta\,(d_t - d_{t-1})\big).$$

Take the depth now, add a fraction β of the last change, never go below zero. β is fitted by least squares on the training part of the training flood only: **0.921** for 30 m and **0.948** for 480 m, a little below 1 so the change is continued but slightly slowed.

The skill score. Error is the average of squared mistakes over all squares, and

$$\text{skill} = 1 - \frac{\text{error of the forecast}}{\text{error of copying the last map}}.$$

Skill 1 is perfect, 0 is no better than copying, below 0 is worse, and 0.9 removes 90% of copying's error.

Worked example. Five squares: depth 30 minutes ago 0, 0.1, 0.4, 0.8, 1.0, now 0, 0.2, 0.5, 1.0, 1.0, what really happens 0, 0.3, 0.6, 1.2, 1.0. *Copying* has mistakes 0, 0.1, 0.1, 0.2, 0 and squares adding to **0.06**. The *trend* with $\beta = 0.9$ gives 0, 0.29, 0.59, 1.18, 1.0, mistakes 0, 0.01, 0.01, 0.02, 0, squares adding to **0.0006**, so skill $= 1 - 0.0006/0.06 = 0.99$. A *bad model* giving 0, 0.5, 0.8, 1.1, 1.1 has squares adding to 0.10 and skill ≈ -0.67, worse than copying (as Model A in late Australia, -1.449).

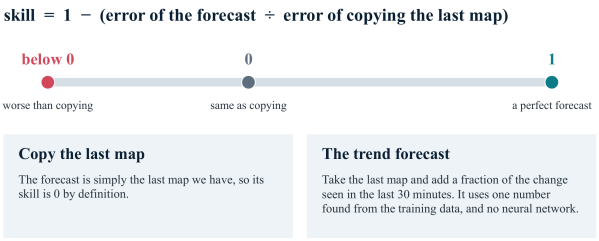


Figure 5. The skill score scale. **Say this:** 0 is copying the last map, 1 is perfect, negative is worse than copying.

Typical error in cm (root mean square error) for copying the last map over the whole map: **0.112 cm (Australia, last 15%), 0.520 (UK), 0.103 (Pakistan, last 15%) and 0.548 (Mozambique)**. These tiny numbers are why copying is such a strong opponent.

Be careful here

Skill in Colab and on the laptop is **not comparable**: Colab scores whole maps 30 minutes ahead, the laptop scores patches containing water from 30 minutes to 2.5 hours.

9 How we tested fairly

Part 1. One model per map size, trained on one flood in time order (70/15/15 as in Chapter 4). Two tests: the **last 15% of the training flood** and the harder **whole never-seen flood**.

Part 2. Three experiments named by map size: **E60** (Australia to UK, 60 m), **E30** (same at 30 m) and **E480** (Pakistan to Mozambique). Each has **three folds**, each holding out a different third of the training region's tiles (a second test, "held-out tiles"). Every Part 2 number is the **mean of three folds**. Every model got the same fixed recipe: Adam, peak learning rate 0.001, 200-step warm-up then cosine decay, gradient clipping at 1.0, loss equal to squared error divided by the squared error of copying, batch 1, 30 passes and 300 training windows (a window is five maps in a row cut from a patch).

Noise and intervals. A **seed** fixes a network's random start. The spread between seeds is **0.006 to 0.010 in skill**, so we treat differences below about **0.02** as noise. A 95% interval (from redrawing the test examples with replacement many times) that does not contain 0 means the difference is probably real.

Hypotheses. Before the final runs we wrote six: **H1** networks beat copying, **H2** they beat the best linear baseline, **H3** depth history matters, **H4** recurrence (LSTM memory) matters, **H5** more rain information helps, **H6** a stricter test gives a similar score. With extra runs this made about **310 training runs**.

Ablations take away one input at test time and see how far the score falls: "rain removed" and "terrain removed" set it to zero, "history frozen" replaces every earlier map with the last one, "history shuffled" puts the earlier maps in the wrong order. If the score hardly changes, the model did not use that input.

10 Part 1: building the forecaster

1. **Input:** four maps 30 minutes apart, three channels each (depth, rain, local ground height).
2. **Two CNN layers** (3 × 3, 16 filters) on each map: water edges, slopes, rain patches.
3. **The ConvLSTM** (3 × 3, 32 filters) reads the four maps in order and keeps a memory map.
4. **A small decoder** (one CNN layer and a last 1 × 1 layer) turns the memory into the *predicted change in depth*.



Figure 6. The forecaster in five boxes. The dark box is the memory (ConvLSTM). **Say this:** maps go through two CNN layers, the ConvLSTM remembers the four steps, and a small decoder draws the change.

Why predict the change, and start at zero? Most of the map hardly changes in 30 minutes, so the network predicts only the change, added to a starting forecast. The last layer **starts at zero**, so before training the output is exactly the starting forecast. A final ReLU keeps depth from going below zero. Every convolution pads, so the map stays 64 × 64.

Table 3. Every layer of Model A for a 64 × 64 tile.

Layer	Output shape	Parameters
Input (depth, rain, local height)	4 × 64 × 64 × 3	0
Convolution 3 × 3, 16 filters, ReLU (each map)	4 × 64 × 64 × 16	448
Convolution 3 × 3, 16 filters, ReLU (each map)	4 × 64 × 64 × 16	2,320
ConvLSTM 3 × 3, 32 filters	64 × 64 × 32	55,424
Convolution 3 × 3, 16 filters, ReLU	64 × 64 × 16	4,624
Convolution 1 × 1, 1 filter (starts at zero)	64 × 64 × 1	17
Add to the starting forecast, then ReLU	64 × 64 × 1	0
Total		**62,833**

Model A and Model B have *exactly the same layers* and differ in the last step. **Model A** starts from the last map and adds the predicted change in metres. **Model B** starts from the trend forecast and adds a small correction in centimetres. We built B after A beat copying but the trend forecast was far better.

Training. 4,000 random 64 × 64 tiles for learning and 600 for validation, Adam at constant learning rate 0.001, batch 16, up to 20 passes, early stopping after 4 passes without improvement (best weights kept), seed 42, mean squared error loss, one model per map size and version, on a Colab GPU.

11 Part 1 results: the surprise

Table 4. Skill against copying the last map, whole maps, 30 minutes ahead (single runs). "Rule" is the water-balance rule below.

Test	Trend	Copy + rule	A	A + rule	B	B + rule
Australia, last 15%	0.990	0.000	−1.449	−0.710	0.989	0.989
UK, never-seen flood	0.947	0.053	0.305	0.320	0.948	0.949
Pakistan, last 15%	0.908	0.142	0.345	0.404	0.908	0.901
Mozambique, never-seen flood	0.942	0.051	0.150	0.172	0.943	0.941

Model A beat copying in three of four tests (0.305, 0.345, 0.150), but the trend forecast with no network scores 0.908 to 0.990, far ahead of Model A in every test. In late Australia Model A scored −1.449. **Model B matches the trend within 0.001 in every test but does not beat it.** A network that beats "copy" can still be bad: ask whether it beats the *best simple forecast*.

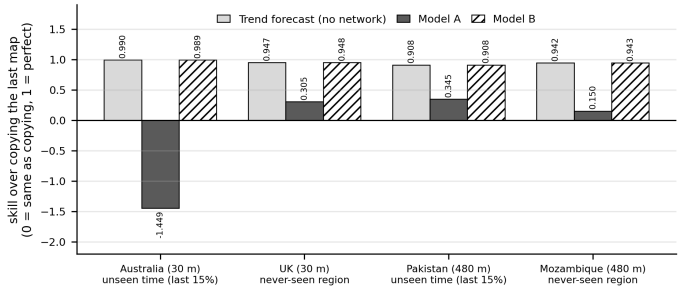


Figure 7. Skill of the trend forecast (no network), Model A and Model B in the four tests.

What went wrong in late Australia. There the water hardly moves and copying is wrong by only about a millimetre, so a model adding even a little water in the wrong place is worse than copying. Our first guess, that the model had only seen water rising, was **wrong**: in 73 of 331 training steps the average depth fell, and the trend number fitted on them scores 0.990 on that last part. The error maps show a faint excess of water over

most of the map at the start of this part, and strong errors for A along rivers and heavy rain, but almost none for B. We think A added a little water almost everywhere, but we did **not** measure this further.

Why B could not do better. Training lowered B's validation error only about 1% (30 m) and 5% (480 m). After the trend only **1% in Australia, 5% in the UK, 9% in Pakistan and 6% in Mozambique** of copying's error is left. For the 30 m Model A the validation error was lowest after the second pass and then rose while practice error kept falling: the overfitting of Chapter 4.

Be careful here

We changed *two* things in Model B at once (starting point and unit of the correction), so we cannot say which made the difference.

The water-balance rule. Almost all the rain stays on the ground, so the average rise of the water in 30 minutes should equal rain (mm per hour) times 0.5 hours (4 mm per hour should raise the average depth by 2 mm). The **water-balance rule** shifts the forecast so its total change matches the last 30 minutes of rain. The data obey it closely (ratio 0.999 Australia, 0.946 UK, 1.028 Pakistan, 1.045 Mozambique, since water leaves at the map edge in three floods and the Indus flows in at Pakistan's edge). The rule **helped Model A in every test**, halving the Australian failure (-1.449 to -0.710), but changed Model B by only -0.007 to $+0.001$. It patched A's weakness and adds no skill to a good forecast.

12 Part 2: the big laptop study

Part 1 left open: which parts of a CNN-LSTM matter? We ran about **310 training runs** on one laptop, up to 2.5 hours ahead. The models: four fitted baselines (linear model of depth history, the same with rain, state and trend, two-feature lookup table, per-pixel MLP), the **original CNN-LSTM** (about 4.5 million parameters), a CNN with a 1×1 kernel (one square only), the **local CNN** (one 5×5 convolution of 16 filters and a 1×1, no memory, 4,101 parameters), a CNN with an 11×11 kernel, the **small ConvLSTM** (8 filters, 4,741 parameters) and a heavy ConvLSTM (652,918, about 140 times larger).

A worrying design in the original CNN-LSTM. About 93% of its parameters sit in two dense layers of $16{,}384 \times 128$ weights, and it squeezes every frame through just **128 numbers** before the LSTM, like describing a photo over the phone: you can say "mostly blue" but not that one pebble moved a millimetre. Its depth input was also divided by 5 m, so a millimetre becomes tiny. We think these choices explain its poor score but did not test this directly.

13 Part 2 results: what really matters

Scores are skill on a **never-seen flood**, mean of three folds, hybrid rain, five maps from 30 minutes to 2.5 hours.

Table 5. Skill on the never-seen flood against copying. Seed noise is about 0.01.

Forecast	E60 (UK)	E30 (UK)	E480 (Moz.)
Lookup table (two features)	0.786	0.763	0.737
Linear model of the depth history	0.818	0.798	0.790
Per-pixel MLP	0.846	0.854	0.829
Original CNN-LSTM	0.187	0.188	0.061
CNN 1×1 kernel / local CNN 5×5 / CNN 11×11	0.848 / 0.841 / 0.841	0.835 / 0.830 / 0.831	0.818 / 0.807 / 0.793
Small ConvLSTM	0.866	0.840	0.811
Heavy ConvLSTM	0.840	not run	0.802

Three findings. (1) **A simple formula does most of the work.** The linear model scores 0.79 to 0.82 and the best networks 0.81 to 0.87, so they add only about 0.01 to 0.05. (2) **Neighbours and size do not help.** Kernels of size 1, 5 and 11 are within 0.025 of each other and a network 140 times larger gains nothing. (3) **The original CNN-LSTM is far behind** at 0.06 to 0.19.

What the models used. Freezing or shuffling the history leaves the original CNN-LSTM at 0.187, so it **ignores the history**, and removing rain drops it to 0.003, so it works almost only from rain. For the other networks **history frozen** takes skill to about zero and **history shuffled** to -2.6 to -3.7, so they depend on the history and its order, while rain removed costs 0.01 to 0.02 and terrain removed 0.006 or less. A local CNN *trained* without depth history falls from 0.84, 0.83 and 0.81 to **0.17, 0.16 and 0.09**. The rain setting matters a little: for the local CNN causal, hybrid and forced rain give 0.822, 0.841, 0.866 (E60), 0.805, 0.830, 0.851 (E30) and 0.794, 0.807, 0.834 (E480).

Does the LSTM memory help? Not when compared fairly. At 30 passes the small ConvLSTM led the local CNN on UK 60 m by $+0.025$ (95% interval $+0.004$ to $+0.043$) and was level on Mozambique. But the plain CNN was undertrained: with 90 passes for both, the difference was $+0.004$ (-0.021 to $+0.027$) on the UK and -0.014 (-0.029 to $+0.006$) on Mozambique, inside the ± 0.03 band fixed in advance as "equivalent". The ConvLSTM looked better in two places (it loses skill more slowly further ahead, and it is steadier in the strict test below), but we did not test that recurrence is the reason.

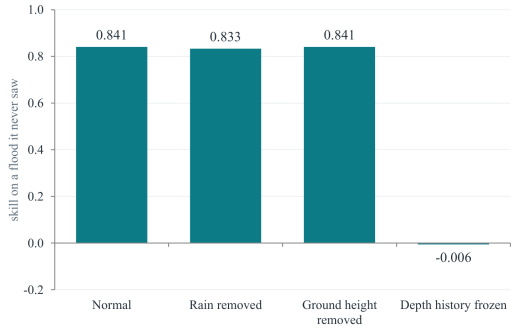


Figure 8. What the networks use. **Say this:** taking away the depth history ruins the forecast, taking away rain or terrain hardly matters.

Tuning. A larger batch (8) with learning rate 3×10^{-3} lowered scores by 0.008 to 0.026. Longer training (90 passes) helped the plain CNN by

0.008 to 0.024 and the ConvLSTM almost nothing. Differences below about 0.02 cannot be resolved.

The strict test. Hiding a whole block of time, including the start of a flood, the plain CNN on hidden Pakistan tiles fell from skill 0.430 to −6.56. Six of 480 windows, all at the very start of the flood, carried **98% of the error**. The ConvLSTM stayed at **0.742** and the lookup table at 0.66. A model that has not seen the start of a flood should not be trusted on one.

Scorecard of the six hypotheses (16 cells in all). H1 held in 13 of 16. H2 held in 15 of 16 (gain 0.02 or more in 8, within noise in 7, linear better by 0.046 in 1). H3 held in all three experiments (a loss of 0.67 to 0.72 without history). H4 mixed: ahead at 30 passes, not supported at 90. H5 held for the local CNN, not for the ConvLSTM. H6 held in only 2 of 5 cells.

14 A mistake we found, and how we fixed it

The laptop linear baselines were **ridge regressions**: weighted sums fitted by least squares with a penalty on big weights. Our inputs were in *metres*, and 30-minute depth changes are tiny (about 0.003), so such an input needs a **big** weight, which the penalty punishes. The penalty squeezed the history weights almost to zero, so two linear baselines **could not use the depth history** and looked far weaker than they are, which made the networks look better than they should.

Aman found it on 10 October 2026. Re-running the old code reproduced the old numbers exactly in all 9 combinations. We then **standardised every input** (same spread) before fitting, and a fit with almost no penalty matched it within 0.001.

Table 6. Skill of the two affected linear baselines (never-seen flood, hybrid rain) before and after the fix.

Linear baseline	E60	E30	E480
Depth history: before / after	0.323 / 0.818	0.345 / 0.798	0.151 / 0.790
With rain, state and trend: before / after	0.345 / 0.820	0.359 / 0.797	0.182 / 0.793

Our claim that the forecast is a *non-linear* function of the depth history is **not supported**, and H2 went from 16 of 16 to **15 of 16**. **Not affected:** the Colab results, H1, H3 to H6, the lookup table, MLP, networks, ablations and tuning. The corrected numbers are those in the final report. Lesson: a weak baseline flatters a model, so check that baselines are strong and fair.

15 Rating the flood: capsule network and written rules

We rate each 64 × 64 tile as **none, minor, moderate or severe**. The **label** is the share of the tile's *normally dry land* under more than 30 cm of water: none below 5%, minor 5% to 15%, moderate 15% to 30%, severe 30% or more. (The earlier laptop label, the single deepest square, let one deep spot decide a whole patch and failed outside Australia at 30 m.) In Pakistan 698 of the 1,000 test tiles are severe, in the UK none.

Capsule network. In an ordinary network every neuron outputs *one number*. A capsule outputs a small *list* (a vector): its **length** says how sure it is that something is there and its **direction** gives details. Capsules in one layer *vote* for capsules in the next, and votes that agree with the others get more weight, **routing by agreement** (like eight friends saying "robin" and two "crow": you trust the eight). Our code repeats it three times. Our capsule head takes the frozen ConvLSTM memory map, shrinks it with three convolutions, forms **256 primary capsules** (8 numbers each), and routes them through **three rounds** to **four class capsules** (16 numbers each), the longest being the answer. The **margin loss** pushes the correct capsule's length above 0.9 and the others below 0.1.

As a control, a plain dense layer on Australia at 30 m (earlier label) scored 60.2% against the capsule's 67.8% without augmentation, but 79.8% against 80.3% with augmentation (rotating and flipping patches, 8 versions), a gap of +0.005 with an interval around zero. A trivial rule on the last observed map scored **97.4%**, so neither network beat a simple threshold.

Table 7. Severity accuracy on 1,000 test tiles per flood (Colab notebook, new label).

Test flood	Capsule alone	Capsule + rules	Class of the forecast map alone
Australia (30 m)	60.9%	67.9%	99.8%
UK (30 m)	50.0%	56.1%	99.1%
Pakistan (480 m)	87.4%	89.1%	99.9%
Mozambique (480 m)	63.5%	70.7%	99.5%

The last column is almost perfect because the rating in 30 minutes is nearly the rating now. **Pakistan's 87.4% is not as good as it looks**, since always answering "severe" is right 69.8% of the time, and 30 m and 480 m accuracies should not be compared (a tile covers about 2 km at 30 m, 30 km at 480 m). A **confusion matrix** is a table of right and wrong answers, with correct ones on the diagonal.

NeuroSymbolic AI joins a neural network ("neuro") with written rules people can read ("symbolic"), so they can check each other. Our physics rule checks the forecast (Chapter 11) and three severity rules check the capsule's answer: **R1** if less than 1% of the land is flooded now and in the forecast, the rating is "none". **R2** if the forecast is not less flooded than now and more than 1 mm per hour of rain fell in the last hour, the rating may not fall below the current level. **R3** if the capsule's and the forecast map's ratings are more than one class apart, use the forecast map's. Every decision comes with a written reason. Of 1,000 tiles the rules changed R1/R2/R3: Australia 49/0/21, UK 0/40/21, Pakistan 0/7/10, Mozambique 134/42/0, and accuracy rose in all four floods (table above). In the laptop study the rule layer even lowered accuracy (80.3% to 74.1%), and its rules and labels are different, so the two cannot be compared.

16 Looking inside: Grad-CAM and explainable AI

A network is a *black box*, and **explainable AI** helps people see what a model is doing. **Grad-CAM** uses the *gradient* of Chapter 4, but about the *answer*: if this part of the memory were turned up a little, how much would the answer move? The ConvLSTM keeps 32 memory maps, each gets a weight from how much it moves the answer, and we add them with those weights, keep the positive part and paint it on the picture. Bright places mattered. For the forecaster the quantity explained is the **total predicted depth**, for the capsule network the **length of the chosen class capsule**. In Australia and the UK the bright areas **follow river channels and flood edges**, in Pakistan large blocks (we think heavy rain) and in Mozambique drainage lines.

Are the heat maps honest? In the laptop study a **deletion test** removed the 10% of cells with the highest heat. For the depth forecast this changed the forecast more than random cells in **84% to 92%** of windows, so it passed. For the severity class the heat maps were weak: the top cells beat

the controls in only 55% of windows, and maps of two seeds agreed with a rank correlation of only **0.17**. We treat the capsule heat maps as illustrations, **not proof**.

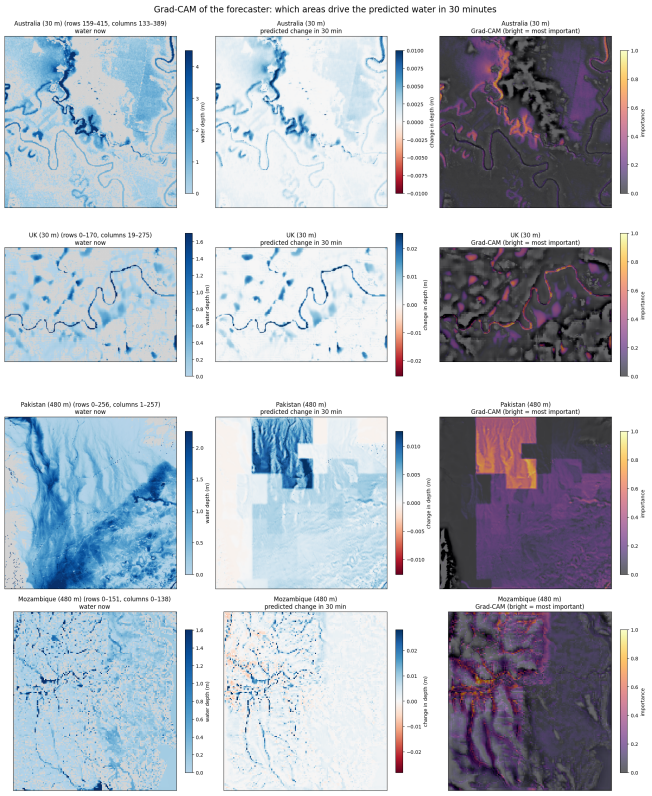


Figure 9. Grad-CAM of the forecaster (Model A): water now, predicted change, heat map over local ground height. **Say this:** the bright places follow rivers and flood edges.

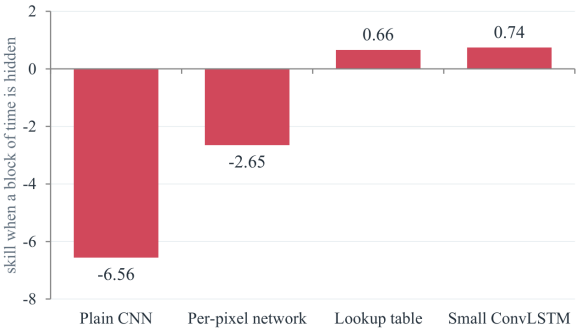


Figure 10. The strict test (start of a flood hidden). **Say this:** the plain CNN breaks down, the ConvLSTM and lookup table stay steady.

Where each course keyword appears: CNN (Chapters 5, 10), LSTM and ConvLSTM (6, 10, 13), capsule network (15), NeuroSymbolic AI (11, 15), explainable AI and Grad-CAM (16).

17 Limits, lessons and what comes next

What this study cannot tell you. Only two independent test pairs (Australia to UK, Pakistan to Mozambique). The floods are simulated. Part 1 results are single runs. The two parts use different protocols and cannot be compared. The capsule network and Grad-CAM were built on Model A, not B. The LSTM memory is untested beyond 2.5 hours. A plain CNN can break completely at the start of a flood it never saw.

Lessons. (1) Compare every model with simple forecasts. (2) A weak baseline flatters a model. (3) Check your own explanations (we first thought Model A never saw draining water, and the data said no). (4) A bigger network is not automatically better. (5) Know your noise: differences below about 0.02 are not claimed. (6) Test on unseen places and hide a block of time too. (7) Write hypotheses down first, and say what you do not know.

Next. Forecast 6 to 24 hours by feeding forecasts back. Split Model B's two changes apart. Rebuild the capsule network and Grad-CAM on Model B. Repeat the Colab runs with several seeds.

Conclusion. We built a CNN-ConvLSTM forecaster for flood depth maps, added a capsule severity network, Grad-CAM and written rules, and tested them on later parts of the training floods and on floods from other countries. Model A beat the easiest reference and looked successful, a second simple reference was far better, and Model B matched it. The laptop study showed the depth history carries nearly all of the skill, and at equal training length the ConvLSTM was no more accurate than a plain CNN, though steadier when the start of a flood was hidden.

18 The key numbers on one page

Part 1 is the Colab notebook and Part 2 the laptop study. The four tests are always in the order Australia, UK, Pakistan, Mozambique.

19 Who did what, and where to find everything

Aman Sumeet Saxena built and ran the Colab work (Part 1) and wrote the three final notebooks (Data exploration, Data preparation, Flood model): data checks, Models A and B, trend forecast, water-balance rule, capsule network, severity rules, Grad-CAM. **Anhad Mahajan** ran the laptop study (Part 2, about 310 training runs), the baselines, ablations, strict tests and the earlier capsule and Grad-CAM tests. The report is joint. Aman found the ridge baseline mistake (Chapter 14) and the baselines were re-fitted with rescaled inputs, with the original code file left unchanged.

Running the notebooks (in this order, outputs are already saved, so for the talk just scroll). (1) *Data exploration* needs about 30 GB of free disk (Colab showed 62.9 GB), downloads all 14,870 files (about 27 GB, 60 minutes in our run) and saves nothing to Drive. (2) *Data preparation* downloads only 3,822 files (9.1 minutes), writes 5.69 GB of arrays to Colab's disk and compares them with the copy on Drive, which it only reads. (3) *Flood model* needs a T4 GPU runtime and loads the prepared data and saved models from Drive. `TRAIN_NEW` and `TRAIN_V2` are False, so nothing is trained again, and True trains again.

Table 8. The numbers to remember.

What	The number
The floods	Four. Learn from Australia and Pakistan, test on the never-seen UK and Mozambique.
Square sizes	30 m (Australia, UK) and 480 m (Pakistan, Mozambique).
Maps kept	One every 30 minutes: 481, 145, 673 and 289 maps.
Part 1 forecaster	62,833 parameters. Four maps in, one map out, 30 minutes ahead.
Error of copying the last map	0.112, 0.520, 0.103 and 0.548 cm.
Trend number β	0.921 for 30 m and 0.948 for 480 m.
Skill of the trend forecast	0.990, 0.947, 0.908 and 0.942.
Skill of Model A	-1.449, 0.305, 0.345 and 0.150.
Skill of Model B	0.989, 0.948, 0.908 and 0.943.
Error left after the trend	1%, 5%, 9% and 6% of the error of copying.
Part 2 size	About 310 training runs. Five maps in, five out, up to 2.5 hours ahead.
Part 2 skill	Lookup table 0.74 to 0.79. Linear model 0.79 to 0.82. Best networks 0.81 to 0.87. Original CNN-LSTM 0.06 to 0.19.
Noise between seeds	0.006 to 0.010, so treat differences below about 0.02 as noise.
A network 140 times larger	No gain (0.840 and 0.802).
Capsule network against a plain layer	67.8% against 60.2%, but 80.3% against 79.8% with augmentation.
Capsule network with the rules (Part 1)	67.9%, 56.1%, 89.1% and 70.7%.
Grad-CAM deletion test	84% to 92% of windows for the depth forecast.
Strict test on hidden Pakistan tiles	Plain CNN 0.430 down to -6.56. ConvLSTM 0.742. Lookup table 0.66.

Table 9. Where things are.

Final report (Word, PDF)	The 28-page report we hand in, the source of the results in this guide.
Final presentation	The 15-slide deck, with the script in the main guide.
Three Colab notebooks	Already run, outputs saved (the main guide).
Laptop study package	The Part 2 code, the roughly 310 result files and the project log.
Ridge baseline check	Scripts, notes and an erratum for the fix of Chapter 14.
Website and README	https://amans-06.github.io/FloodCastBench-Project/ and the GitHub repository FloodCastBench-Project.

20 A to Z of new words

Ablation. Removing one input or part to see how much the score falls.
Adam. A method that adjusts the learning step during training.
Array. A grid of numbers in numpy, with a shape.
Augmentation. Extra training examples made by rotating and flipping.
Baseline. A simple forecast a model must beat.
Batch. Examples looked at before each weight nudge.
Capsule. A unit giving a short list of numbers, whose length says how sure it is.
Capsule network. Capsules that vote for each other (routing by agreement).
Causal rain. Only rain that has already fallen.
Cell. One box of a Colab notebook, code or writing.
Channel. One layer of a map (depth, rain, height).
CNN. Convolutional network, slides small filters over a map.
Colab. Free online notebook that lends a GPU.
Confusion matrix. Table of right and wrong answers of a classifier.
ConvLSTM. An LSTM whose memory is a map and whose gates use filters.
Copy forecast. Next map equals last map (persistence).
CPU and GPU. Normal processor and graphics chip, the GPU is much faster for networks.
DataFrame. A pandas table with named columns.
Deletion test. Removing the hottest heat-map cells to see if the forecast changes more than for random cells.
Dense layer. Every neuron sees every number of the layer before.
E60, E30, E480. Part 2 experiments: Australia to UK at 60 m and 30 m, Pakistan to Mozambique at 480 m.
Early stopping. Stop when the mock-test score stops improving.
Epoch. One pass through all training examples.
Explainable AI. Tools that show what a model is doing.
FABDEM. Ground height map used by the simulator.
Filter. Small window of weights sliding over a map.
FloodCastBench. Public dataset of four simulated floods.
Fold. One of three rounds with a different third of tiles held out.
Forced rain. The next five rain maps are given too.
Frozen. Weights not changed in training.
GPM-IMERG. Satellite rain data used by the simulator.
Grad-CAM. Heat map of which places mattered for an answer.
Gradient. Slope telling which way to change a weight to lower the loss.
Grid. A map made of squares holding numbers.
Hugging Face. Website that stores datasets and models.
Hybrid rain. The original input, with up to 30 minutes of rain not yet fallen.
Hypothesis. A guess we test, written down before the final runs.
Keras, TensorFlow. Free libraries for building networks.
Kernel. Another word for filter.
Label. The right answer used to train a classifier.
Learning rate. Size of each weight step.
Linear model. A weighted sum of inputs.
Lookup table. Average answer looked up for each pair of values.
Loss. How wrong the network is, learning makes it small.
LSTM. Long short-term memory, with forget, write and read gates.
Margin loss. Capsule loss: correct capsule above 0.9, others below 0.1.
Mean squared error. Average of squared mistakes.
MLP. Multilayer perceptron, small network of dense layers.
Model A, Model B. Part 1 forecasters, A starts from the last map, B from the trend.
Neural network. Many simple neurons with learned weights.
NeuroSymbolic AI. A network checked by written rules.
Never-seen flood. Flood hidden during training (UK, Mozambique).
Notebook. Colab file of cells, outputs stay saved.
NPY file. One saved numpy array.
Overfitting. Memorising practice examples and failing on new ones.
Parameter. A number the network learns, such as a weight.
Pass. One run through all training examples.
Pixel. One square of a picture or map.
Rank correlation. How alike two maps order places, 1 same, near 0 none.
Rasterio. Library that reads map files (tif) into arrays.
ReLU. Turns negative numbers into zero.
Ridge regression. Linear fit with a penalty keeping weights small.
RMSE. Root mean square error, typical mistake in the original unit.
Routing by agreement. Capsules whose votes agree get more weight.
Seed. Number that fixes the random start so a run repeats.
Shape. Size of an array in each direction, e.g. 4, 64, 64, 3.
Skill. 1 minus forecast error divided by copy error.
Spatiotemporal. In both space and time.
Squash. Shrinks a vector to length 0 to 1, keeping its direction.
Strict split. Test where a block of time is also hidden from training.
Tile or patch. Small square cut from a big map.
Token. Secret key (Hugging Face) kept in Colab secrets.
Training, validation, test. Practice questions, mock test, final exam.
TRAIN_NEW. Model notebook switch: False loads saved models, True trains again.
Trend forecast. Last map plus a fraction of the last 30-minute change.
Water-balance rule. Shifts a forecast so its total rise matches the rain.
Weight. Number a neuron multiplies an input by.
Window. Part 2 example: a 128 by 128 patch with its maps in time.